Prepared for self-study · August 2026

Table of Contents

1. Why Microservices Matter at This Stage
   - 1.1 What interviewers expect from 3–5 YOE candidates
   - 1.2 How this guide is organized
2. Monolith to Microservices
   - 2.1 Why teams move to microservices (and when not to)
   - 2.2 The Strangler Fig migration pattern
3. Core Design Principles
   - 3.1 Bounded Context (Domain-Driven Design)
   - 3.2 Single Responsibility at service granularity
   - 3.3 Database-per-service
   - 3.4 Sizing a service correctly
4. Service Communication Patterns
   - 4.1 Synchronous communication
   - 4.2 Asynchronous communication
   - 4.3 Choosing sync vs async
   - 4.4 The chatty-services trap
5. Service Discovery
   - 5.1 Why discovery is needed
   - 5.2 Client-side vs server-side discovery
   - 5.3 Eureka, Consul, Kubernetes DNS
6. API Gateway
   - 6.1 Responsibilities of a gateway
   - 6.2 Backend for Frontend (BFF)
7. Configuration & Secrets Management
   - 7.1 Externalized configuration
   - 7.2 Secrets management
8. Data Management Patterns
   - 8.1 Database-per-service, in depth
   - 8.2 Saga pattern
   - 8.3 Outbox pattern
   - 8.4 CQRS
   - 8.5 Event Sourcing
9. Resilience & Fault Tolerance

Microservices

A Practical Guide for Java Backend Developers

Target Audience: 3–5 Years Experience

Topic-wise, simple-language explanations of microservices architecture — design, communication, data, resilience, and operations — with Spring Boot/Spring Cloud specifics and interview tips.

9.1 Circuit Breaker
9.2 Retry with backoff
9.3 Bulkhead & Rate Limiter
9.4 Timeouts
9.5 Resilience4j in Spring Boot

10. Observability

10.1 Correlation IDs & distributed tracing
10.2 Centralized logging
10.3 Metrics & health checks

11. Security

11.1 Authentication at the edge (OAuth2/JWT)
11.2 Service-to-service auth
11.3 Defense in depth

12. Containers & Orchestration

12.1 Docker basics for a Spring Boot service
12.2 Kubernetes concepts you need to know

13. Service Mesh

13.1 What a mesh adds over application code
13.2 When it's worth adopting

14. Testing Strategies

14.1 The testing pyramid for microservices
14.2 Contract testing
14.3 Testcontainers for integration tests

15. CI/CD & Deployment Strategies

15.1 Pipeline stages
15.2 Blue-green, canary, rolling deployments

16. Anti-Patterns & Common Pitfalls

16.1 The distributed monolith
16.2 Other frequent mistakes

17. Spring Cloud Ecosystem Cheat Sheet

17.1 Which Spring Cloud project solves which problem

18. Interview Approach & Worked Case Study

18.1 How to answer "design a microservices system"
18.2 Case study: decomposing an e-commerce platform

1. Why Microservices Matter at This Stage

1.1 What interviewers expect from 3–5 YOE candidates

By 3–5 years, most Java backend developers have worked on at least one service that talks to others over the network. Interviewers at this level want to see that you understand microservices as a set of *trade-offs*, not a default best practice. They expect you to:

- Explain what problem splitting a system into services actually solves (team autonomy, independent scaling/deployment) — and what it costs (operational complexity, network unreliability, data consistency challenges).
- Know the concrete patterns used to manage that complexity: service discovery, circuit breakers, sagas, distributed tracing — not just the buzzwords.
- Recognize a "distributed monolith" — services split by name but still tightly coupled — as an anti-pattern, not a valid design.
- Connect the theory to Spring Boot/Spring Cloud tools you'd actually reach for.

> **Simple way to think about it:** Microservices trade one hard problem (a large, tangled codebase) for a different hard problem (a distributed system with network calls, partial failures, and eventual consistency). It's worth it only when the first problem is genuinely worse than the second for your team and product.

1.2 How this guide is organized

Each topic explains the concept in plain language, why it exists, a short example or Spring-specific note, and pitfalls to watch for. Interview tip boxes highlight what to emphasize when the topic comes up.

2. Monolith to Microservices

2.1 Why teams move to microservices (and when not to)

Good reasons	Bad reasons
Different parts of the system need to scale independently (e.g. checkout vs product catalog)	"Everyone else is doing it"
Multiple teams need to deploy independently without blocking each other	Team is small (1–2 squads) and doesn't have the ops maturity yet
Different modules have genuinely different technology needs	Hoping it will magically fix a messy codebase (it won't — a messy monolith becomes a messy distributed system)
Clear, stable domain boundaries already exist	Domain boundaries are still unclear/changing rapidly

Simple rule of thumb: "Monolith first" is still good advice for most new products. Split into services once you have real evidence of scaling limits or team-coordination pain — not before.

2.2 The Strangler Fig migration pattern

Named after a vine that gradually grows around a tree and eventually replaces it. Instead of a risky "rewrite everything at once," you incrementally carve functionality out of the monolith into new services, routing traffic for that functionality through a facade/gateway to the new service, while the rest still goes to the monolith. Over time, more and more of the monolith is "strangled" until little or nothing remains.

- **Step 1:** Put a routing layer (API Gateway or reverse proxy) in front of the monolith.
- **Step 2:** Pick one well-bounded capability (e.g. "notifications") and extract it into a new service.
- **Step 3:** Update the routing layer to send notification-related requests to the new service; everything else still goes to the monolith.
- **Step 4:** Repeat for the next capability. The monolith shrinks incrementally, with a working system at every step.

Interview tip: If asked "how would you migrate a legacy monolith to microservices," the Strangler Fig pattern is the expected answer — emphasize incremental, low-risk extraction over a "big bang" rewrite, which is a well-known source of failed migrations.

3. Core Design Principles

3.1 Bounded Context (Domain-Driven Design)

A **bounded context** is a boundary within which a particular domain model and its terminology are consistent. The word "Order" might mean something different to the Sales team than to the Shipping team — each should own its own model of "Order" within its own bounded context, rather than forcing one shared "Order" model across the whole company. Microservice boundaries should align with bounded contexts, not with technical layers (e.g. not "one service for all database access").

> **Simple analogy:** "Account" means something different to a bank's Fraud team (a risk profile) than to its Statements team (a transaction ledger). Each team should be free to model "Account" in the way that fits their own concerns, rather than sharing one rigid definition everyone has to compromise around.

3.2 Single Responsibility at service granularity

Just as a class should have one reason to change (SRP, from the Design Patterns guide), a service should own one cohesive business capability — "Payments," "Inventory," "Notifications" — not an arbitrary technical slice like "all the REST controllers."

3.3 Database-per-service

Each microservice owns its own database, and no other service is allowed to access it directly — all access goes through the owning service's API. This is what actually enables independent deployability: if Service B could read Service A's tables directly, changing Service A's schema would risk breaking Service B, recreating the tight coupling microservices are meant to remove.

> **Pitfall:** A shared database across multiple services is the single most common way teams accidentally build a "distributed monolith" (Section 16.1) — you get all the operational complexity of microservices with none of the deployment independence.

3.4 Sizing a service correctly

There's no fixed line count or team size rule. A useful heuristic: a service should be small enough that one small team can own it end-to-end (including on-call), and its data/behavior should change together as a unit. If two "services" almost always need to be deployed together to work correctly, they're probably one service that's been split prematurely.

4. Service Communication Patterns

4.1 Synchronous communication

REST gRPC Request/response

The caller waits for a response before proceeding. Simple to reason about and debug, but couples the availability of the caller to the availability of the callee — if the downstream service is slow or down, the caller is affected too (unless resilience patterns from Section 9 are applied).

REST is the default for most service APIs; **gRPC** (binary, Protobuf-based, supports streaming) is preferred for internal, high-throughput, latency-sensitive service-to-service calls where the performance and strong typing outweigh REST's simplicity and tooling ubiquity.

4.2 Asynchronous communication

Message queues Events Pub-sub

The caller publishes a message/event and moves on without waiting for the receiver to process it. Decouples services in time (they don't need to be online simultaneously) and improves resilience (a temporary outage in the consumer doesn't block the producer). The trade-off is added complexity: eventual consistency, harder debugging, and the need for idempotent consumers (since most brokers guarantee at-least-once delivery).

Kafka is typically used for high-throughput event streams and where consumers may want to replay history; RabbitMQ is often preferred for task-queue-style workloads and complex routing rules.

4.3 Choosing sync vs async

Use synchronous when…	Use asynchronous when…
The caller genuinely needs the result before continuing (e.g. "is this payment valid?")	The caller doesn't need to wait for downstream processing to finish (e.g. "send a confirmation email")
Low latency matters more than resilience to downstream outages	Decoupling and resilience matter more than immediate consistency
The interaction is simple request/response	Multiple independent services need to react to the same event

4.4 The chatty-services trap

If handling a single user request requires Service A to call B, which calls C, which calls D synchronously, latency compounds (each hop adds its own network round-trip) and a single slow/failing hop can cascade into a full failure. Watch for this during design: it's often a sign that

service boundaries were drawn along the wrong lines (Section 3.1), or that some of that data should be denormalized/cached locally instead of fetched live on every request.

5. Service Discovery

5.1 Why discovery is needed

In a dynamic environment, service instances scale up/down and get new IPs constantly (especially in containers/Kubernetes). Hardcoding IP addresses or hostnames doesn't work — services need a way to find healthy instances of each other dynamically.

5.2 Client-side vs server-side discovery

- **Client-side discovery** — the calling service queries a registry directly, gets back a list of healthy instances, and picks one itself (often with client-side load balancing). More logic in the client, but no extra network hop.
- **Server-side discovery** — the caller sends the request to a well-known address (a load balancer/proxy), which looks up and forwards to a healthy instance. Simpler clients, but an extra hop and a component that must itself be highly available.

5.3 Eureka, Consul, Kubernetes DNS

Tool	Model	Notes
Netflix Eureka	Client-side discovery	Classic Spring Cloud choice; largely superseded by Kubernetes-native approaches in newer stacks
Consul	Supports both models	Also does health checking, KV config storage, and service mesh integration
Kubernetes DNS + Services	Server-side discovery	The default in any Kubernetes-hosted system — a Service gets a stable DNS name, kube-proxy routes to healthy Pods automatically

> **Java angle:** In a Spring Cloud app, `@LoadBalanced RestTemplate` or `WebClient` combined with Spring Cloud LoadBalancer lets you call another service by its logical name (e.g. `http://inventory-service/items`) instead of a hardcoded host/port — discovery and load balancing happen transparently underneath.

6. API Gateway

6.1 Responsibilities of a gateway

A single entry point in front of all microservices, handling concerns that would otherwise be duplicated in every service:

- **Routing** — directing requests to the right backend service.
- **Authentication** — validating tokens once at the edge instead of in every service.
- **Rate limiting** — protecting backend services from abusive or excessive traffic.
- **Request/response transformation** — adapting a legacy service's response shape for newer clients, or aggregating multiple backend calls into one client-facing response.
- **Cross-cutting observability** — a natural place to inject correlation IDs (Section 10.1) into every request.

Common choices: Spring Cloud Gateway (Java-native, reactive), Kong, NGINX, cloud-managed gateways (AWS API Gateway).

6.2 Backend for Frontend (BFF)

A variation where, instead of one generic gateway serving all clients, you build a dedicated gateway per client type (a Web BFF, a Mobile BFF) — each shaped around what that specific client actually needs, avoiding a single gateway trying to be everything to everyone. Useful when mobile and web clients have meaningfully different data/latency needs.

> **Pitfall:** Piling business logic into the gateway (beyond routing/auth/ rate-limiting) turns it into a hidden monolith that every service secretly depends on. Keep the gateway thin; business logic belongs in the services.

7. Configuration & Secrets Management

7.1 Externalized configuration

Configuration (DB URLs, feature flags, timeouts) should live outside the deployed artifact, so the same build can be promoted across dev/staging/prod without rebuilding. Spring Cloud Config Server (backed by a Git repo) is the classic Spring choice; Kubernetes ConfigMaps are the container-native equivalent. Both let you change configuration without redeploying code.

7.2 Secrets management

Passwords, API keys, and certificates should never live in plain config files or source control. Dedicated secret stores (HashiCorp Vault, AWS Secrets Manager, Kubernetes Secrets combined with encryption at rest) inject secrets at runtime and support rotation without a code change or redeploy.

> **Pitfall:** A surprisingly common real-world incident source: a secret committed to Git "just for now" during local testing, then never removed — Git history retains it even after a later commit deletes it. Use a secrets manager from day one, even in small projects.

8. Data Management Patterns

This is usually the hardest part of microservices in practice — once data is split across services, maintaining consistency without a shared database transaction requires deliberate patterns.

8.1 Database-per-service, in depth

Recall Section 3.3: each service owns its data exclusively. This means a single business operation that used to be one ACID database transaction (e.g. "reserve inventory AND charge payment") now spans multiple services and multiple databases — you can no longer wrap it in one transaction. This is precisely the problem the next three patterns solve.

8.2 Saga pattern

Breaks a cross-service business transaction into a sequence of local transactions, each with a **compensating action** to undo it if a later step fails.

- **Choreography** — each service publishes an event when it finishes its step; the next service reacts to that event. No central coordinator, but harder to see the overall flow as the number of steps grows.
- **Orchestration** — a central saga orchestrator explicitly tells each service what to do next and handles compensation logic itself. Easier to reason about and monitor, at the cost of a new central component.

```
Order Saga (orchestrated):
  1. Reserve inventory        → on failure: (nothing to compensate yet)
  2. Charge payment           → on failure: release inventory
  3. Schedule shipment        → on failure: refund payment, release inventory
  4. Mark order confirmed
```

8.3 Outbox pattern

Problem: A service needs to update its own database AND publish an event about that change — but doing both isn't atomic by default. If the DB write succeeds and the message broker publish fails (or vice versa), data and events drift out of sync (the "dual write" problem).

Solution: Write the event to an "outbox" table in the *same local database transaction* as the business data change (so both succeed or fail together, guaranteed by the DB's own ACID transaction). A separate background process (or Debezium-style change-data-capture) reads the outbox table and reliably publishes those events to the message broker, marking them as sent.

> **Java angle:** Spring's `@Transactional` around the business write and the outbox-row insert makes this straightforward — both go through the same `EntityManager` /transaction, so they commit or roll back together.

8.4 CQRS

Command Query Responsibility Segregation — separates the write model (handles commands that change state) from the read model (optimized for queries), often with different underlying data stores or schemas per side. In a microservices context, this is often used when a service's read patterns (fast lookups by many different criteria) are very different from its write patterns (a small number of well-defined state transitions).

8.5 Event Sourcing

Instead of storing only current state, every state change is stored as an immutable event; current state is derived by replaying events. Gives a complete audit trail and the ability to reconstruct past state, at the cost of added complexity (event schema evolution, snapshotting for performance). Often paired with CQRS, but usable independently. Use it deliberately — not every service needs a full audit trail badly enough to justify the complexity.

> **Interview tip:** If asked how to keep data consistent across services, the Saga pattern (with the Outbox pattern for reliable event publishing) is the expected, realistic answer. Mentioning 2PC (two-phase commit) as something you'd deliberately avoid at this scale — and explaining why (blocking, poor availability under partition) — shows depth.

9. Resilience & Fault Tolerance

In a distributed system, partial failure is the normal case, not the exception. These patterns keep one failing service from taking down everything that depends on it.

9.1 Circuit Breaker

Stops a service from repeatedly calling a downstream dependency that's already failing. States: **Closed** (calls flow normally) → **Open** (after too many failures, calls fail fast without even attempting the call) → **Half-Open** (after a cooldown, a few test calls check if the dependency has recovered).

9.2 Retry with backoff

Automatically retries a failed call, using exponential backoff (waiting longer between each attempt) and jitter (random variation) to avoid many clients retrying in lockstep and overwhelming a recovering service. Only retry operations that are safe to repeat — i.e. idempotent ones (see the System Design guide, Section 4.3).

9.3 Bulkhead & Rate Limiter

Bulkhead isolates resources (e.g. separate thread pools per downstream dependency) so a slow dependency can't exhaust the resources needed to serve calls to a healthy one. **Rate limiter** caps how many calls a service makes to (or accepts from) a dependency in a given time window, protecting both sides from overload.

9.4 Timeouts

Every inter-service network call must have an explicit timeout. Without one, a single slow dependency can tie up threads/connections indefinitely, and that exhaustion can cascade backward through every service that depends on the caller.

9.5 Resilience4j in Spring Boot

```
@CircuitBreaker(name = "inventoryService", fallbackMethod = "fallbackCheckStock")
@Retry(name = "inventoryService")
@TimeLimiter(name = "inventoryService")
public CompletableFuture<Boolean> checkStock(String sku) {
    return CompletableFuture.supplyAsync(() -> inventoryClient.checkStock(sku));
}

private CompletableFuture<Boolean> fallbackCheckStock(String sku, Throwable t) {
    return CompletableFuture.completedFuture(false); // graceful degradation
}
```

Resilience4j (the modern replacement for the now-deprecated Netflix Hystrix) provides all of the above as composable annotations, each independently configurable per named dependency via `application.yml`.

10. Observability

You cannot operate a system you cannot see across. Observability matters more, not less, once a single user request spans multiple services.

10.1 Correlation IDs & distributed tracing

A **correlation ID** (or trace ID) is generated when a request first enters the system (usually at the API Gateway) and propagated through every downstream call (as an HTTP header, or a message header for async calls). Every log line and trace span across every service includes this ID, letting you reconstruct the full path of one request across a dozen services. Tools: Zipkin, Jaeger, or OpenTelemetry (the emerging vendor-neutral standard, increasingly the default choice for new systems).

> **Java angle:** Spring Cloud Sleuth (now integrated into Micrometer Tracing in newer Spring Boot versions) automatically generates and propagates trace/span IDs through `RestTemplate` / `WebClient` calls and Kafka messages, and injects them into your log output with almost no manual wiring.

10.2 Centralized logging

Each service writes structured (JSON) logs to stdout; a log shipper (Fluentd, Filebeat, or the cloud provider's native agent) forwards them to a central store (ELK stack, or a managed equivalent) where they can be searched and correlated by trace ID across services. Never rely on SSH-ing into individual containers to read logs — containers are ephemeral and logs disappear with them.

10.3 Metrics & health checks

Spring Boot Actuator exposes `/actuator/health` (used by Kubernetes liveness/readiness probes to restart or remove unhealthy instances automatically) and `/actuator/prometheus` (scraped by Prometheus, visualized in Grafana) with almost no configuration. At minimum, track request rate, error rate, and latency percentiles per service — the "RED" metrics (Rate, Errors, Duration) commonly used as the starting dashboard for any microservice.

11. Security

11.1 Authentication at the edge (OAuth2/JWT)

Clients authenticate once (typically via an OAuth2 flow) and receive a JWT access token. The API Gateway validates this token at the edge; downstream services can trust that a request reaching them has already been authenticated, and typically just need to check the token's claims for authorization (what this caller is allowed to do), not re-verify identity from scratch each time.

11.2 Service-to-service auth

Even internal, service-to-service calls should be authenticated — not implicitly trusted just because they're "inside the network." Common approaches: propagating the original user's JWT downstream, issuing separate service-identity tokens (client credentials OAuth2 flow), or mutual TLS (mTLS), where both sides present certificates to verify each other's identity, often handled transparently by a service mesh (Section 13).

11.3 Defense in depth

Don't rely solely on the gateway for security. Apply rate limiting, input validation, and authorization checks within each service too — the gateway can be misconfigured, bypassed, or simply not cover every path (e.g. internal service-to-service calls that don't go through it at all).

12. Containers & Orchestration

12.1 Docker basics for a Spring Boot service

Each microservice is packaged as a container image — a self-contained bundle of the app, its JVM, and its dependencies, guaranteeing it runs the same way everywhere. A typical Spring Boot Dockerfile uses a multi-stage build (build the JAR in one stage with the full JDK/Maven toolchain, then copy just the JAR into a slim JRE-only runtime image) to keep the final image small.

```
FROM eclipse-temurin:21-jre-alpine
COPY target/app.jar app.jar
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

12.2 Kubernetes concepts you need to know

Concept	What it is
Pod	The smallest deployable unit — one or more tightly coupled containers sharing network/storage
Deployment	Manages a set of identical Pod replicas, handles rolling updates and self-healing
Service	A stable network endpoint/DNS name routing to a set of healthy Pods (server-side discovery, Section 5.3)
ConfigMap / Secret	Externalized configuration and sensitive values injected into Pods at runtime
Liveness / Readiness probe	Health checks Kubernetes uses to restart unhealthy Pods or hold traffic until a Pod is ready
HorizontalPodAutoscaler	Automatically scales the number of Pod replicas based on CPU/memory/custom metrics

13. Service Mesh

13.1 What a mesh adds over application code

A service mesh (Istio, Linkerd) deploys a lightweight proxy ("sidecar") alongside every service instance, intercepting all network traffic in and out. This lets it provide, uniformly and without application code changes: mTLS between services, retries/timeouts/circuit breaking at the infrastructure level, fine-grained traffic routing (e.g. canary releases, Section 15.2), and rich network-level observability — all consistently applied across every service, regardless of which language or framework it's written in.

13.2 When it's worth adopting

A service mesh adds real operational complexity (another system to run and understand). It tends to pay off once you have dozens of services and the inconsistency of each team implementing resilience/security patterns independently in application code becomes a real problem. For a handful of services, application-level libraries (Resilience4j, Spring Security) are usually simpler and sufficient.

14. Testing Strategies

14.1 The testing pyramid for microservices

- **Unit tests** — fast, isolated, the majority of your tests, no network/DB involved.
- **Integration tests** — test a service against real (or realistic) infrastructure (its own DB, message broker) using Testcontainers (Section 14.3).
- **Contract tests** — verify that a service's API still matches what its consumers expect, without needing to spin up the consumer or provider together (Section 14.2).
- **End-to-end tests** — the fewest, slowest, most brittle; exercise a full user flow across multiple real services. Keep these to a small set of critical paths.

> **Simple rule of thumb:** The pyramid shape matters — many fast unit tests at the base, fewer integration tests, fewer still end-to-end tests at the top. Inverting this (few unit tests, many slow end-to-end tests) is a very common source of slow, flaky CI pipelines.

14.2 Contract testing

Problem: Service A (consumer) depends on Service B's (provider) API. If B changes its API in a way that breaks A, you want to catch that in CI — before deployment — without needing to run both services together in every test.

Solution: The consumer defines a "contract" (expected request/response shape); the provider's CI pipeline replays that contract against its actual implementation and fails the build if it no longer matches. Pact is the most widely used tool for this pattern, often described as "consumer-driven contract testing."

14.3 Testcontainers for integration tests

A Java library that spins up real dependencies (PostgreSQL, Kafka, Redis) in Docker containers just for the duration of a test run, then tears them down — giving you integration tests against the real technology instead of a mock or an in-memory substitute that might not behave the same way in production.

```
@Testcontainers
class OrderRepositoryTest {
    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:16");

    @Test
    void savesAndFindsOrder() { /* real Postgres, no mocking */ }
}
```

15. CI/CD & Deployment Strategies

15.1 Pipeline stages

A typical microservice pipeline: build & unit test → static analysis/security scan → build container image → integration/contract tests → deploy to staging → automated smoke tests → deploy to production (using one of the strategies below). Each service should have its own independent pipeline, so one team's deploy is never blocked by another team's code.

15.2 Blue-green, canary, rolling deployments

Strategy	How it works	Trade-off
Rolling	Old instances replaced with new ones gradually, a few at a time	Simple, default in Kubernetes Deployments; brief mixed-version period
Blue-green	Full new environment ("green") deployed alongside the old ("blue"); traffic switched over all at once	Instant rollback (switch back to blue); needs double the infrastructure briefly
Canary	New version rolled out to a small % of traffic first, monitored, then gradually increased	Limits blast radius of a bad deploy; needs good metrics/automation to detect issues quickly

16. Anti-Patterns & Common Pitfalls

16.1 The distributed monolith

Services that are technically separate processes but are still tightly coupled — sharing a database (Section 3.3), requiring synchronized deployments, or communicating so chattily (Section 4.4) that they can't actually be understood, tested, or deployed independently. You end up with all the operational cost of microservices (network calls, more moving parts to monitor) and none of the benefits (independent scaling, independent deployment). This is the single most common real-world microservices failure mode.

16.2 Other frequent mistakes

- **Premature decomposition** — splitting a system into services before domain boundaries are well understood, leading to constant, painful re-drawing of service boundaries later.
- **Shared libraries for business logic** — a shared JAR containing domain logic used by multiple services re-creates coupling through a different mechanism; a version bump now requires coordinating multiple service deployments.
- **No ownership of cross-cutting failure modes** — nobody explicitly designs for "what happens when Service B is down," so it's discovered for the first time during an actual incident.
- **Ignoring data consistency until it's a production issue** — treating Section 8's patterns as optional rather than a required part of the design for any operation spanning services.
- **Too many services, too small** — "nanoservices" where the overhead of a network call and separate deployment pipeline outweighs any real benefit of separation.

17. Spring Cloud Ecosystem Cheat Sheet

17.1 Which Spring Cloud project solves which problem

Concern	Spring project
Service discovery	Spring Cloud Netflix Eureka (or native Kubernetes discovery)
API Gateway	Spring Cloud Gateway
Externalized configuration	Spring Cloud Config
Client-side load balancing	Spring Cloud LoadBalancer
Resilience (circuit breaker/retry/bulkhead)	Resilience4j (via Spring Cloud Circuit Breaker abstraction)
Distributed tracing	Micrometer Tracing (successor to Spring Cloud Sleuth) + Zipkin/OpenTelemetry
Messaging abstraction	Spring Cloud Stream (Kafka/RabbitMQ binders)
Metrics	Spring Boot Actuator + Micrometer (feeds Prometheus/Grafana)
OAuth2/JWT security	Spring Security OAuth2 Resource Server / Authorization Server

18. Interview Approach & Worked Case Study

18.1 How to answer "design a microservices system"

1. **Clarify scope** — what does the system need to do, what scale, what consistency requirements per operation.
2. **Identify bounded contexts** — propose service boundaries based on business capability, not technical layers (Section 3.1).
3. **Design communication** — for each interaction between services, justify sync vs async (Section 4.3).
4. **Address data consistency explicitly** — call out which operations span services and how you'd keep them consistent (Saga/Outbox, Section 8).
5. **Layer in cross-cutting concerns** — gateway, discovery, resilience, observability, security — and explain where each lives.
6. **Call out trade-offs and failure modes** — what happens when each service is down; what's eventually consistent vs strongly consistent, and why that's an acceptable choice for this domain.

18.2 Case study: decomposing an e-commerce platform

Proposed services (by bounded context)

- **Catalog Service** — product data, search; read-heavy, cached aggressively.
- **Inventory Service** — stock levels; needs strong consistency for stock reservation.
- **Order Service** — order lifecycle, orchestrates the checkout saga.
- **Payment Service** — talks to external payment providers; isolated for compliance (PCI scope reduction) as much as for technical reasons.
- **Notification Service** — email/SMS/push, consumes events asynchronously.

Key decisions

- Checkout is an orchestrated Saga (Section 8.2) run by Order Service: reserve inventory → charge payment → confirm order → publish `OrderConfirmed` event.
- Notification Service subscribes to `OrderConfirmed` asynchronously (Section 4.2) — the customer shouldn't wait for an email to send before their order confirms.
- Catalog reads are served from a Redis cache in front of Catalog Service, since browsing traffic vastly exceeds purchase traffic (recall the read:write ratio heuristic from the System Design guide).
- Each service publishes its domain events via the Outbox pattern (Section 8.3) to avoid the dual-write problem between its database and the message broker.
- All external traffic enters through an API Gateway that handles JWT validation and rate limiting once, at the edge (Section 6.1, 11.1).

Failure modes called out

If Payment Service is down mid-saga, the compensating action releases the inventory reservation made in step 1, and the order is marked failed rather than left in an inconsistent "reserved forever" state. If Notification Service is down, order confirmation still succeeds (it doesn't block on notifications) and queued events are processed once the service recovers — graceful degradation (Section 9) rather than a cascading failure.

End of guide · Every pattern here exists to manage a cost that splitting into services introduces — know the cost before reaching for the pattern.